

UNIT - II

SYLLABUS

Distributed mutual exclusion : Requirement of mutual exclusion algorithm, Non token based algorithms: Lamport's algorithm, Ricart-Agrawala algorithm, Mackawa's algorithm, token based algorithms: Suzuki-Kasami's broadcast algorithm, Singhal's heuristic algorithm, Raymond's tree-based algorithm, comparative performance analysis.

INTRODUCTION

In the problem of mutual exclusion, concurrent access to a shared resource by several un-co-ordinated user requests is serialized to secure the integrity of the shared resource. Mutual exclusion is a fundamental issue in the design of distributed systems and an efficient and robust technique for mutual exclusion is essential to the viable design of distributed systems.

The problem of mutual exclusion becomes much more complex in distributed systems because of the lack of both shared memory and a common physical clocks and because of unpredictable message delays. Owing to these factors, it is virtually impossible for a site in a distributed system to have current and complete knowledge of the state of the system.

DISTRIBUTED MUTUAL EXCLUSION

Q.1. What do you mean by distributed mutual exclusion? Explain.

Ans. Distributed mutual exclusion :

- Mutual exclusion is a fundamental problem in distributed computing systems. Mutual exclusion ensures that concurrent access of processes to a shared resource or data is serialized, that is, executed in a mutually exclusive manner.
- Mutual exclusion in a distributed system states that only one process is allowed to execute the critical section (CS) at any given time.
- In a distributed system, shared variables (semaphores) or a local kernel cannot be used to implement mutual exclusion.
- Message passing is the sole means for implementing distributed mutual exclusion. The decision as to which process is allowed access to the CS next is arrived at by message passing, in which each process learns about the state of all other processes in some consistent way.

- The design of distributed mutual exclusion algorithms is complex because these algorithms have to deal with unpredictable message delays and incomplete knowledge of the system state.
- There are two basic approaches for implementing distributed mutual exclusion :
 - (1) Token-based approach.
 - (2) Non token-based approach.
- In the token-base approach, a unique token (also known as the PRIVILEGE message) is shared among the sites. A site is allowed to enter its CS if it possesses the token and it continues to hold the token until the execution of the CS is over.
- Mutual exclusion is ensured because the token is unique. The algorithms based on this approach essentially differ in the way a site carries out the search for the token.
- In the non-token-based approach, two or more successive rounds of messages are exchanged among the sites to determine which site will enter the CS next.
- A site enters the critical section (CS) when an assertion, defined on its local variables, becomes true. Mutual exclusion is enforced because the assertion becomes true only at one site at any given time.

Q.2. Explain the system model of mutual exclusion in distributed operating system.

Ans. System model :

- The system consists of N sites, $S_1, S_2, \dots, S_N$. Without loss of generality, we assume that a single process is running on each site. The process at site S_i is denoted by p_i . All these processes

communicate asynchronously over an underlying communication network.

- A process wishing to enter the CS requests all other or a subset of processes by sending REQUEST messages and waits for appropriate replies before entering the CS.
- While waiting the process is not allowed to make further requests to enter the CS, executing the CS, or neither requesting nor executing the CS (i.e. idle).
- In the "requesting the CS" state, the site is blocked and cannot make further requests for the CS.
- In the "idle" state, the site is executing outside the CS. In the token-based algorithms, a site can also be in a state where a site holding the token is executing outside the CS. Such state is referred to as the idle token state.
- At any instant, a site may have several pending requests for CS. A site queues up these requests and serves them one at a time.
- We do not make any assumption regarding communication channels if they are FIFO or not. This is algorithm specific. We assume that channels reliably deliver all messages, sites do not crash and the network does not get partitioned.
- Some mutual exclusion algorithms are designed to handle such situations. Many algorithms use Lamport-style logical clocks to assign a timestamp to critical section requests. Timestamps are used to decide the priority of requests in case of a conflict.
- The general rule followed is that the smaller the timestamp of a request, the higher its priority to execute the CS.
- We use the following notation : N denotes the number of processes or sites involved in invoking the critical section, T denotes the average message delay and E denotes the average critical section execution time.

REQUIREMENTS OF MUTUAL EXCLUSION ALGORITHM

Q.3. What are the requirements of mutual exclusion algorithm?

Ans. Requirements of mutual exclusion algorithm :

- The primary objective of a mutual exclusion algorithm is to maintain mutual exclusion; that is, to guarantee that only one request accesses the CS at a time.

- The following characteristics are considered important in a mutual exclusion algorithm:

(1) Freedom from deadlocks:

Two or more sites should not endlessly wait for messages that will never arrive.

(2) Freedom from starvation:

A site should not be forced to wait indefinitely to execute CS while other sites repeatedly executing CS. That is, every requesting site should get an opportunity to execute CS in a finite time.

(3) Fairness:

- Fairness dictates that requests must be executed in the order they are made (or the order in which they arrive in the system).
- Since a physical global clock does not exist, time is determined by logical clocks. Fairness implies freedom from starvation but not vice-versa.

(4) Fault tolerance :

A mutual exclusion algorithm is fault-tolerant if in the wake of a failure, it can recognize itself so that it continues to function without any disruptions.

Q.4. How to measure the performance of mutual exclusion algorithm?

Ans. Performance measurement of mutual exclusion algorithm :

- The performance of mutual exclusion algorithms is generally measured by the following four metrics : First, the number of messages necessary per CS invocation. Second, the synchronization delay, which is the time required after a site leaves the CS and before the next site enters the CS (see Fig. (a)).
- Note that normally one or more sequential message exchanges are required after a site exits the CS and before the next site enters the CS. Third, the response time, which is the time interval a request waits for its CS execution to be over after its request messages have been sent out (see Fig. (b)).
- Thus, response time does not include the time a request waits at a site before its request messages have been sent out. Fourth, the system throughput, which is the rate at which the system executes requests for the CS.

- If sd is the synchronization delay and E is the average critical section execution time, then the throughput is given by the following equation :

$$\text{System throughput} = \frac{1}{(sd + E)}$$

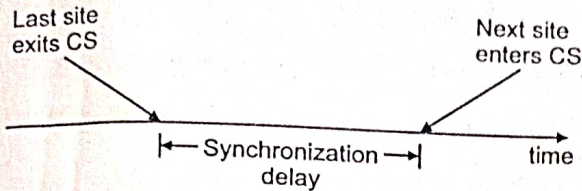


Fig. (a) Synchronization delay

Low and high load performance :

- Performance of a mutual exclusion algorithm depends upon the loading conditions of the system and is often studied under two special loading conditions, viz. low load and high load.
- Under low load conditions, there is seldom more than one request for mutual exclusion simultaneously in the system.
- Under high load conditions, there is always a pending request for mutual exclusion at a site. Thus, after having executed a request, a site immediately initiates activities to let the next site execute its CS.
- A site is seldom in an idle state under high load conditions. For many mutual exclusion algorithms, the performance metrics can be easily determined under low and high loads through simple reasoning.

Best and worst case performance :

- Generally, mutual exclusion algorithms have best and worst cases for the performance metrics. In the best case, prevailing conditions are such that a performance metric attains the best possible value.
- For example, in most algorithms the best value of the response time is a round trip message delay plus CS execution time, $2T + E$ (where T is the average message delay and E is the average critical section execution time).
- Often for mutual exclusion algorithms, the best and worst cases coincide with low and high loads, respectively.
- For example, the best and worst values of the response time are achieved when the load is, respectively, low and high.
- The best and the worse message traffic is generated in Maekawa's

algorithm at low and high load conditions, respectively. When the value of a performance metric fluctuates statistically, we generally talk about the average value of that metric.

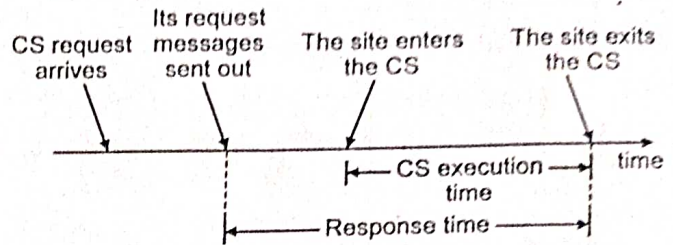


Fig. (b) Response time

NON TOKEN BASED ALGORITHMS

Q.5. Write a short note on non-token based algorithms.

Ans. Non-token based algorithms :

- It is also called as permission based algorithm.
- In non-token-based mutual exclusion algorithms, a site communicates with a set of other sites to arbitrate who should execute the CS next.
- For a site S_i , request set R_i contains ids of all those sites from which site S_i must acquire permission before entering the CS (Critical Section).
- These algorithms use timestamp to order requests for the CS and to resolve conflict between simultaneous requests for the CS.
- In all these algorithm, logical clock are maintained and updated according to Lamport's scheme.
- Each request for CS gets timestamp and smaller timestamp requests have priority over larger timestamp requests.
- Non token based algorithms includes: Lamport's Algorithm, Ricard-Agrawala Algorithm, Maekawa's Algorithm.

LAMPORT'S ALGORITHM

Q.6. Explain Lamport's algorithm with the help of suitable example.

OR Briefly explain Lamport's algorithm.

Ans. Lamport's algorithm :

- Lamport was the first to give a distributed mutual exclusion algorithm as an illustration of his clock synchronization scheme.

- Let R_i be the request set of site S_i , i.e. the set of sites from which S_i needs permission when it wants to enter CS.
- In Lamport's algorithm, $\forall i: 1 \leq i \leq N:: R_i = \{S_1, S_2, \dots, S_N\}$. Every site S_i keeps a queue, $request_queue_i$, which contains mutual exclusion requests ordered by their timestamps.
- This algorithm requires messages to be delivered in the FIFO order between every pair of sites.

Algorithm :

(I) Requesting the critical section :

- When a site S_i wants to enter the CS, it sends REQUEST (ts_i, i) message to all the sites in its request set R_i and places the request on $request_queue_i$ (ts_i is the timestamp of the request).
- When a site S_j receives the REQUEST (ts_i, i) message from site S_i , it returns a timestamped REPLY message to S_i and places site S_i 's request on $request_queue_j$.

(II) Executing the critical section

Site S_i enters the CS when the two following conditions hold :

- [L1 :] S_i has received a message with timestamp larger than (ts_i, i) from all other sites.
- [L2 :] S_i 's request is at the top $request_queue_i$.

(III) Releasing the critical section :

- Site S_i , upon exiting the CS, removes its request from the top of its request queue and sends a timestamped RELEASE message to all the sites in its request set.
- When a site S_j receives a RELEASE message from site S_i , it removes S_i 's request from its request queue.

When a site removes a request from its request queue, its own request may come at the top of the queue, enabling it to enter CS. The algorithm executes CS requests in the increasing order of timestamps.

Correctness :

Theorem: Lamport's algorithm achieves mutual exclusion.

Proof:

- Proof is by contradiction. Suppose two sites S_i and S_j are executing the CS concurrently. For this to happen, conditions L_1 and L_2 must hold at both the sites concurrently.
- This implies that at some instant in time, say t , both S_i and S_j have their own requests at the top of their request queues and condition L_1 holds at them. Without loss of generality, assume that S_i 's request has smaller timestamp than the request of S_j .
- From condition L_1 and FIFO property of the communication channels, it is clear that at instant t the request of S_i must be present in $request_queue_j$ when S_j was executing its CS.
- This implies that S_j 's own request is at the top of its own request queue when a smaller timestamp request, S_i 's request, is present in the $request_queue_j$ – a contradiction.

Example :

- In Fig. (1) through Fig. (4), we illustrate the operation of Lamport's algorithm.
- In Fig. (1) site S_1 and S_2 are making request for CS and send out REQUEST message to other site. The timestamps of the message are (2, 1) and (1, 2) respectively.

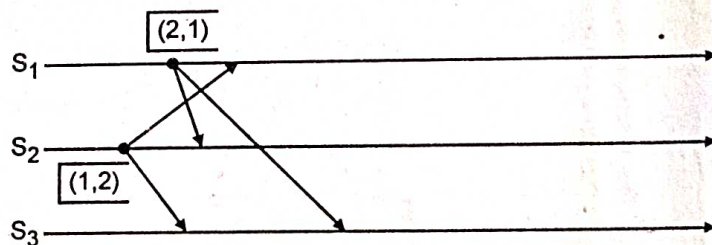
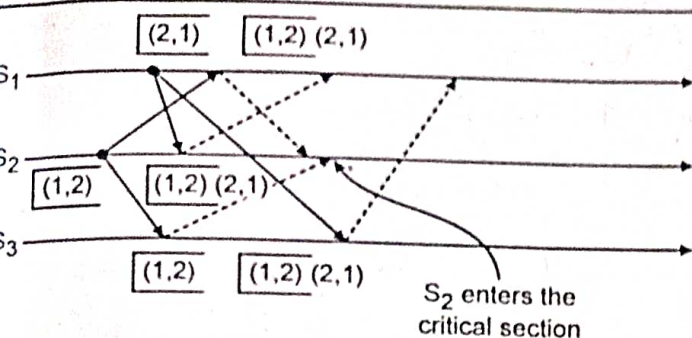
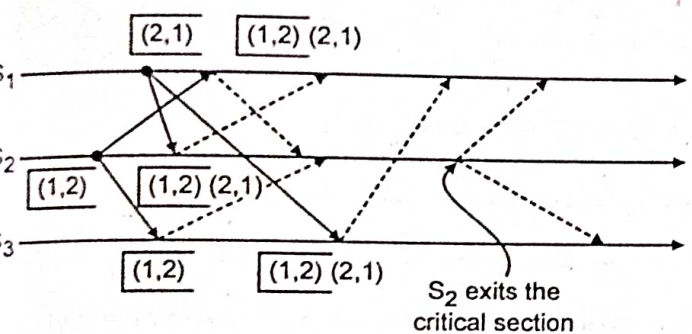


Fig. (1) Site S_1 and S_2 are making requests for the CS

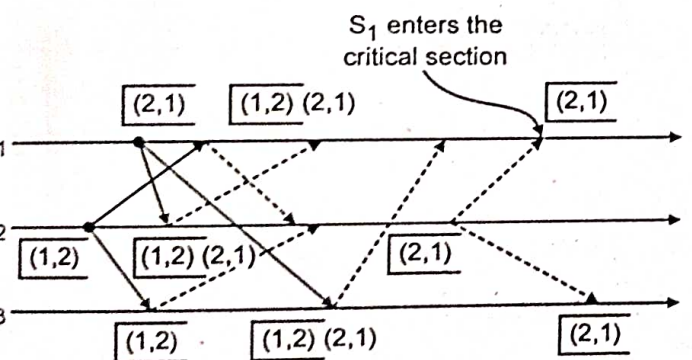
- In Fig.(2) S_2 has received REPLY message from all the other sites and its request is at the top of the $request_queue$. Consequently, it enters the CS.

Fig. (2) Site S_2 enters the CS

In Fig (3) S_2 exits and sends RELEASE message to all other sites.

Fig. (3) Site S_2 exits the CS and send RELEASE messages

In Fig (4) site S_1 has receive REPLY message from all other sites and its request is at top of its request_queue. Consequently, it enters the CS next.

Fig. (4) Site S_1 enter the CS

Performance :

For each CS execution, Lamport's algorithm requires $(N - 1)$ REQUEST messages, $(N - 1)$ REPLY messages and $(N - 1)$ RELEASE messages.

Thus, Lamport's algorithm requires $3(N - 1)$ messages per CS invocation.

Synchronization delay in the algorithm is T .

Optimization :

Lamport's algorithm can be optimized to require between $3(N - 1)$ and $2(N - 1)$ messages per CS execution by pending messages in certain situation.

- For example, suppose site S_j receives a REQUEST message from site S_i after it has sent its own REQUEST message with timestamp higher than the timestamp of site S_i 's request.
- In this case, site S_j need not send a REPLY message to site S_i . This is because when site S_i receive site S_j 's request with a timestamp higher than its own, it can conclude that site S_j does not have any smaller timestamp request that is still pending (because the communication) medium preserves message ordering).

RICART-AGRAWALA'S ALGORITHM

Q.7. Discuss Ricart-Agrawala's Algorithm in detail.

Ans. Ricart-Agrawala's Algorithm :

- The Ricart-Agrawala algorithm is an optimization of Lamport's algorithm that dispenses with RELEASE messages by cleverly merging them with REPLY messages.

- In this algorithm also,

$$\forall i; 1 \leq i \leq N : R_i = \{S_1, S_2^*, \dots, S_N\}$$

Algorithm :

(I) Requesting the critical section :

- When a site S_i wants to enter the CS, it sends a timestamped REQUEST message to all the sites in its request set.
- When site S_j receives a REQUEST message from site S_i , it sends a REPLY message to site S_i if site S_j is neither requesting nor executing the CS or if site S_j is requesting and S_i 's request's timestamp is smaller than S_j 's own request's timestamp. The request is otherwise deferred.

(II) Executing the critical section :

Site S_i enters the CS after it has received REPLY messages from all the sites in its request set.

(III) Releasing the critical section :

- When site S_i exits the CS, it sends REPLY messages to all the deferred requests.

- A site's REPLY messages are blocked only by sites that are requesting the CS with higher priority (i.e. a smaller timestamp). Thus, when a site sends out REPLY messages to all the deferred requests, the site with the next highest priority request receives the last needed REPLY message and enters the CS. The execution of CS requests in this algorithm is always in the order of their timestamps.

Correctness :

Theorem: The Ricart-Agrawala algorithm achieves mutual exclusion.

Proof :

- Proof is by contradiction. Suppose two sites S_i and S_j are executing the CS concurrently and S_i 's request has higher priority than the request of S_j . Clearly, S_i received S_j 's request after it has made its own request.
- Thus, S_j can concurrently execute the CS with S_i only if S_i returns a REPLY to S_j (in response to S_j 's request) before S_i exits the CS.
- However, this is impossible because S_j 's request has lower priority. Therefore, Ricart-Agrawala algorithm achieves mutual exclusion.

Example :

- Fig.(1) through Fig. (4) illustrates the operation of the Ricart-Agrawala algorithm.
- In Fig.(1), site S_1 and S_2 are making request for CS, sending out REQUEST messages to other sites.

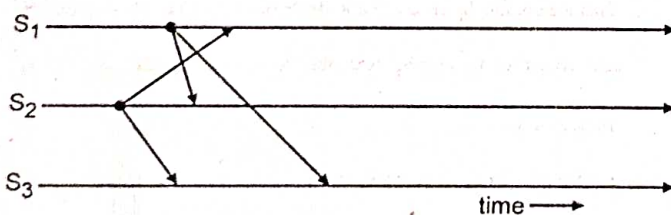


Fig. (1) Site S_1 and S_2 are making requests for the CS

- The timestamps of the request are (2, 1) and (1, 2) respectively.
- In Fig.(2), S_2 has received REPLY message from all other sites and consequently, it enters the CS.

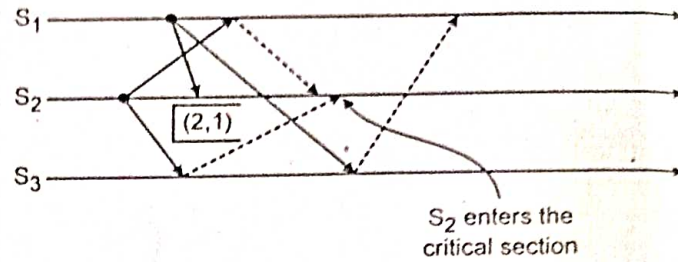


Fig. (2) Site S_2 enters the CS

- In Fig.(3), site S_2 exits the CS and send REPLY message to site S_1 .

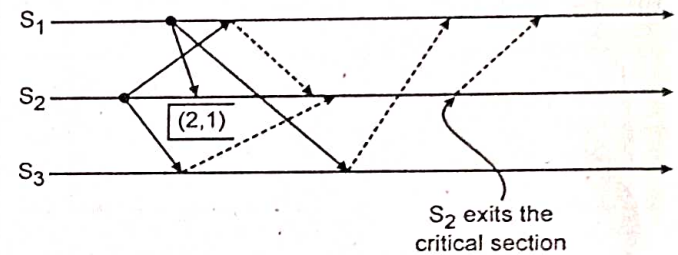


Fig. (3) Site S_2 exits the CS and send REPLY message to S_1

- In Fig.(4), site S_1 has received REPLY messages from all other sites and enters the CS next.

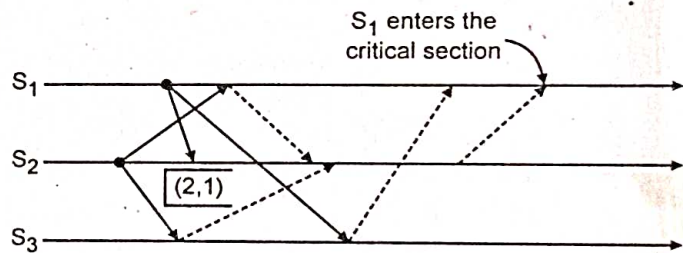


Fig. (4) Site S_1 enters the CS

Performance :

- For each CS execution, Ricart-Agrawala algorithm requires $(N - 1)$ REQUEST messages and $(N - 1)$ REPLY messages.
- Thus, it requires $2(N - 1)$ messages per CS execution.
- Synchronization delay in the algorithm is T .

Optimization :

- The improvement to the Ricart-Agrawala algorithm by observing that once a site S_i has received a REPLY message from a site S_j , the authorization implicit in the message remains valid until S_i sends a REPLY message to S_j (which happens only after the reception of a REQUEST message from S_j).

Therefore, after site S_i has received a REPLY message from site S_j , site S_i can enter its CS any number of times without requesting permission from site S_j until S_i sends a REPLY message to S_j .

With this change, a site in the Ricart-Agrawala's algorithm requests permission from dynamically varying set of sites and requires 0 to 2 (N-1) message per CS execution.

MAEKAWA'S ALGORITHM

Q.8. Explain Maekawa's algorithm in detail.

Ans. Maekawa's algorithm :

In Maekawa's algorithm, a site does not request permission from all the sites, but only from a subset of sites.

In Maekawa's algorithm, the request set of each site is chosen such that :

$$\forall_i \forall_j : 1 \leq i, j \leq N :: R_i \cap R_j \neq \emptyset.$$

Consequently, every pair sites has a site that mediates conflicts between that pair.

In Maekawa's algorithm a site can send out only one REPLY message at a time.

A site can only send a REPLY message only after it has received a RELEASE message for the previous REPLY message. Therefore, a site S_i locks all the sites in R_i in exclusive mode before executing its CS.

The construction of request sets :

The request sets for sites in Maekawa's algorithm are constructed to satisfy the following conditions:

$$M1: (\forall_i \forall_j : i \neq j, 1 \leq i, j \leq N :: R_i \cap R_j \neq \emptyset).$$

$$M2: (\forall_i : 1 \leq i \leq N :: S_i \in R_i)$$

$$M3: (\forall_i : 1 \leq i \leq N :: |R_i| = K)$$

$$M4: \text{Any site } S_j \text{ is contained in } K \text{ number of } R_i\text{'s, } 1 \leq i, j \leq N.$$

Maekawa established the following relation between N and K :

$$N = K(K-1) + 1. \text{ This relation gives } |R_i| = \sqrt{N}.$$

Algorithm :

(I) Requesting the critical section :

(1) A site S_i requests access to the CS by sending REQUEST(i) messages to all the sites in its request set R_i .

(2) When a site S_j receives the REQUEST(i) message, it sends a REPLY(j) message to S_i provided it hasn't sent a REPLY message to a site from the time it received the last RELEASE message. Otherwise, it queues up the REQUEST for later consideration.

(II) Executing the critical section :

Site S_i accesses the CS only after receiving REPLY messages from all the sites in R_i .

(III) Releasing the critical section :

(1) After the execution of the CS is over, site S_i sends RELEASE(i) message to all the sites in R_i .

(2) When a site S_j receives a RELEASE(i) message from site S_i , it sends a REPLY message to the next site waiting in the queue and deletes that entry from the queue. If the queue is empty, then site updates its state to reflect that the site has not sent out any REPLY message.

Correctness :

Theorem:

Maekawa's algorithm achieves mutual exclusion.

Proof :

- Proof is by contradiction. Suppose two sites S_i and S_j are concurrently executing the CS.
- This means site S_i received a REPLY message from all sites in R_i and concurrently site S_j was able to receive a REPLY message from all sites in R_j .
- If $R_i \cap R_j = \{S_k\}$, then site S_k must have sent REPLY messages to both S_i and S_j concurrently, which is a contradiction.

Performance :

- The execution of a CS requires $\sqrt{N}$ REQUEST, $\sqrt{N}$ REPLY and $\sqrt{N}$ RELEASE messages, resulting in $3\sqrt{N}$ messages per CS execution.

- Synchronization delay in this algorithm is $2T$. This is because after a site S_i exits the CS, it first releases all the sites in R_i and then one of those sites sends a REPLY message to the next site that executes the CS.

Problem of deadlocks :

- Mackawa's algorithm can deadlock because a site is exclusively locked by other sites and requests are not prioritized by their timestamps.
 - Assume three sites S_i , S_j and S_k simultaneously invoke mutual exclusion.
 - Suppose $R_i \cap R_j = \{S_{ij}\}$, $R_j \cap R_k = \{S_{jk}\}$ and $R_k \cap R_i = \{S_{ki}\}$.
 - Since sites does not send REQUEST message to the sites in their request sets in any particular order, it is possible that due to arbitrary message delays:
- S_{ij} has been locked by S_i (forcing S_j to wait at S_{ij}).
 - S_{jk} has been locked by S_j (forcing S_k to wait at S_{jk}).
 - S_{ki} has been locked by S_k (forcing S_i to wait at S_{ki}).
- This state represents a deadlock involving sites S_i , S_j and S_k .

Handling deadlocks :

- Mackawa's algorithm handles deadlocks by requiring a site to yield a lock if the timestamp of its request is larger than the timestamp of some other request waiting for the same lock.
 - A site suspects a deadlock (and initiates message exchanges to resolve it) whenever a higher priority request arrives and waits at a site because the site has sent a REPLY message to a lower priority request.
 - Deadlock handling requires three types of messages:
- (1) **FAILED** : A FAILED message from site S_i to site S_j indicates that S_i cannot grant S_j 's request because it has currently granted permission to a site with a higher priority request.
 - (2) **INQUIRE** : An INQUIRE message from S_i to S_j indicates that S_i would like to find out from S_j if it has succeeded in locking all the sites in its request set.

- (3) **YIELD** : A YIELD message from site S_i to S_j indicates that S_i is returning the permission to S_j (to yield to a higher priority request at S_j).

Mackawa's algorithm handles deadlocks as follows:

- When a REQUEST(ts, i) from site S_i blocks at site S_j because S_j has currently granted permission to site S_k , then S_j sends a FAILED(j) message to S_i if S_i 's request has lower priority. Otherwise, S_j sends an INQUIRE(j) message to site S_k .
- In response to an INQUIRE(j) message from site S_j , site S_k sends a YIELD(k) message to S_j provided S_k has received a FAILED message from a site in its request set or if it sent a YIELD to any of these sites, but has not received a new GRANT from it.
- In response to a YIELD(k) message from site S_k , site S_j assumes as if it has been released by S_k , places the request of S_k at appropriate location in the request queue and sends a GRANT(j) to the top request's site in the queue.
- Thus, Mackawa's algorithm requires extra messages to handle deadlocks and may exchange these messages even though there is no deadlock. The maximum number of messages required per CS execution in this case is $5\sqrt{N}$.

Q.9. Explain the generalized non-token based algorithm.

Ans. Generalized non-token based algorithm :

- In information structure of a mutual exclusion algorithm defines the data structure needed at a site to record the status of other sites. The information kept in the information structure is used by a site in making decisions (i.e. from which sites to request permission) when invoking mutual exclusion.
- The information structures at a site S_i consists of the following three sets : a request set R_i , an inform set I_i and a status set St_i . These sets consists of the ids of the sites of the system.

- A site must acquire permission from all the sites in its request set before entering CS. Every site must inform all the sites in its inform set of its status change due to the wait to enter the CS and due to the exit from the CS.
- The status set St_i contains the ids of sites for which S_i maintains status information.
- Note that the inform set and the status set are dependent on each other. The contents of one is decided by the contents of the other. If $S_i \in I_j \Rightarrow S_j \in St_i$.
- A site also maintains a variable CSSTAT, which indicates the site's knowledge of the status of the CS. Every site maintains a queue which contains REQUEST messages, in the order of their timestamps, for which no GRANT message has been sent.

Correctness conditions :

To guarantee mutual exclusion, the information structure of sites in the generalized algorithm must satisfy the conditions given by the following theorem.

Theorem :

If $\forall i: 1 \leq i \leq N :: S_i \in I_i$, then the following two conditions are necessary and sufficient to guarantee mutual exclusion :

$$G1 : \forall i: 1 \leq i \leq N :: I_i \subset R_i$$

$$G2 : \forall i \forall j: 1 \leq i, j \leq N :: (I_i \cap I_j \neq \emptyset) \vee (S_i \in R_j \wedge S_j \in R_i)$$

The correctness condition G2 states that for every pair of sites, either they request permission from each other or they request permission from a common site (which maintains the status information of both).

The generalized algorithm :

In the generalized algorithm, each request for a CS is assigned a timestamp that is maintained according to Lamport's scheme. Timestamps are used to prioritize requests in case of conflicts.

(I) Requesting the critical section :

- (1) To execute CS, a site sends timestamped REQUEST messages to all the sites in its request set.
- (2) On the receipt of a REQUEST message, a site S_i takes the following actions :
 - It places the request in its queue (which is ordered by timestamps).

- If CSSTAT indicates that the CS is free, then it sends a GRANT message to the site at the top of the queue and removes its entry from the queue. If the recipient of the GRANT message is in St_i , then CSSTAT is set to indicate that the site is in CS.

(II) Executing the critical section :

A site executes the CS only after it has received a GRANT message from all the sites in its request set.

(III) Releasing the critical section :

On exiting the CS, the site sends a RELEASE message to every site in its inform set. On receiving a RELEASE message, a site S_i takes the following actions :

- CSSTAT is set to free.
- If its queue is nonempty, then it sends a GRANT message to the site at the top of the queue and removes its entry from the queue. If the recipient of the GRANT message is in St_i , then CSSTAT is set to indicate that the site is in the CS.
- The previous action is repeated until CSSTAT indicates that a site is in the CS or its queue becomes empty.
- The proof of correctness is quite involved and is therefore omitted.
- In the generalized algorithm, a site S_i acquires permission of the Maekawa type from all the sites in its inform set I_i and acquires permission of the Ricart-Agrawala type from all the sites in set $R_i - I_i$.
- In response to a REQUEST message, a site S_j sends Maekawa type permission to sites in its status set and sends Ricart-Agrawala type permission to all the other sites.
- After a site has granted Maekawa type permission, it cannot grant permission to any other site unless it has been released.
- However, in the generalized algorithm a site can concurrently grant many Ricart-Agrawala type permission preceding a Maekawa type permission.
- If the first predicate of correctness condition G2 is false for all S_i and S_j , then the resulting algorithm is of the Ricart-Agrawal type.

- If the second predicate of condition G2 is false for all S_i and S_j , then the resulting algorithm is of the Maekawa type.
- If the first predicate is true for some sites and the second predicate is true for other sites, then the resulting algorithm is a generalized mutual exclusion algorithm.

Example :

Fig. illustrates three mutual exclusion algorithms in terms of their information structures. A solid arrow from S_i to S_j indicates that

$S_j \in I_i$ and $S_j \in R_i$.

- A dashed arrow from S_i to S_j indicates that $S_j \in R_i$ and $S_j \notin I_i$.

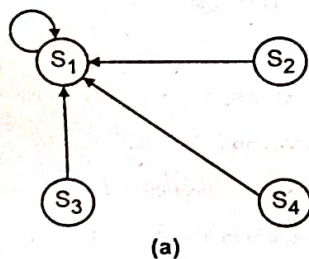
Fig (a) shows the information structure of a mutual exclusion algorithm in which a single site, S_1 , controls entry into the CS [3].

- Fig (b) shows the information structure of a mutual exclusion algorithm where every site requests permission of every other site to enter the CS and a site maintains information about its own status.

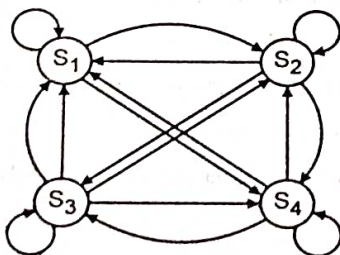
- An example of such an algorithm is the Ricart-Agrawala algorithm.

Fig. (c) shows the information structure of Maekawa's mutual exclusion algorithm with 4 sites. Note that in this case, the request set and the inform set of every site is identical and

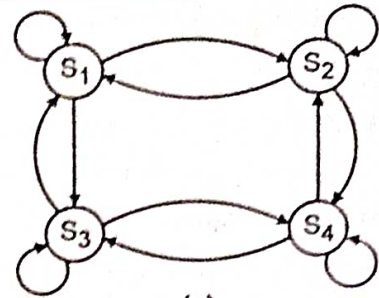
$$\forall i \forall j: i \neq j, 1 \leq i, j \leq 4 :: R_i \cap R_j \neq \phi.$$



(a)



(b)



(c)

Fig. Examples of information structures

TOKEN BASED ALGORITHMS

Q.10. Write a short note on token based algorithms.

Ans. Token-based algorithms :

- In token-based algorithms, a unique token is shared among all sites. A site is allowed to enter its CS if it possesses the token.
- Depending upon the way a site carries out its search for the token there are numerous token-based algorithms used such as :

- Suzuki-Kasami's broadcast algorithm.
- Singhal's heuristic algorithm.
- Raymonds tree based algorithm.

The token based algorithms works on the two factors :

- First, token-based algorithms use sequence numbers instead of timestamps. Every request for the token contains a sequence number and the sequence numbers of sites advance independently.
- A site increments its sequence number counter every time it makes a request for the token.
- A primary function of the sequence numbers is to distinguish between old and current requests.
- Second, a correctness proof of token-based algorithms to ensure that mutual exclusion is enforced is trivial because an algorithm guarantees mutual exclusion so long as a site holds the token during the execution of the CS. Rather, the issues of freedom from starvation and freedom from deadlock are prominent.

SUZUKI-KASAMI'S BROADCAST ALGORITHM

Q.11. Explain Suzuki-Kasami's broadcast algorithm.

Ans. Suzuki-Kasami's broadcast algorithm :

- In the Suzuki-Kasami's algorithm, if a site attempting to enter the

A site that does not have the token, it broadcasts a REQUEST message for the token to all the other sites. A site that possesses the token sends it to the requesting site upon receiving its REQUEST message.

If a site receives a REQUEST message when it is executing the CS, it sends the token only after it has exited the CS.

A site holding the token can enter its CS repeatedly until it sends the token to some other site.

The main design issues in this algorithm are:

- (1) Distinguishing outdated REQUEST messages from current REQUEST message and
- (2) Determining which site has an outstanding request for the CS.

Outdated REQUEST messages are distinguished from current REQUEST message in the following manner:

A REQUEST message of site S_j has the form REQUEST (j, n) where, n ($n = 1, 2, \dots$) is a sequence number that indicates that site S_j is requesting its n^{th} CS execution.

- A site S_i keeps an array of integers $RN_i[1..N]$ where, $RN_i[j]$ is the largest sequence number received so far in a REQUEST message from site S_j . A REQUEST (j, n) message received by the site S_i is outdated if $RN_i[j] > n$. When site S_i receives a REQUEST (j, n) message, it sets $RN_i[j] := \max(RN_i[j], n)$.

Sites with outstanding requests for the CS are determined in the following manner:

The token consists of a queue of requesting sites, Q and an array of integers $LN[1..N]$ where $LN[j]$ is the sequence number of the request that site S_j executed most recently.

- After executing its CS, a site S_i updates $LN[i] := RN_i[i]$ to indicate that its request corresponding to sequence number $RN_i[i]$ has been executed.
- The token array $LN[1..N]$ permits a site to determine if some other site has an outstanding request for CS.
- Note that at site S_i if $RN_i[j] = LN[j] + 1$, then site S_j is currently requesting the token. After having executed the CS, a site checks this condition for all the j 's to determine all the sites that are

requesting the token and places their ids in queue Q if not already present in this queue Q . Then the site sends the token to the site at the head of the queue Q .

The algorithm:

(I) Requesting the critical section:

- (1) If the requesting site S_j does not have the token, then it increments its sequence number, $RN_j[i]$, and sends a REQUEST(j, s_n) message to all other sites (s_n is the updated value of $RN_j[i]$).
- (2) When a site S_j receives this message, it sets $RN_j[i]$ to $\max(RN_j[i], s_n)$. If S_j has the idle token, then it sends the token to S_j if $RN_j[i] = LN[j] + 1$.

(II) Executing the critical section:

Site S_i executes the CS when it has received the token.

(III) Releasing the critical section:

Having finished the execution of the CS, site S_i takes the following actions:

- (1) It sets $LN[i]$ element of the token array equal to $RN_i[i]$.
- (2) For every site S_j whose ID is not in the token queue, it appends its ID to the token queue if $RN_j[j] = LN[j] + 1$.
- (3) If token queue is nonempty after the above update, then it deletes the top site ID from the queue and sends the token to the site indicated by the ID.

- Thus, after executing its CS, a site gives priority to other sites with outstanding requests for the CS.

- The Suzuki-Kasami algorithm is not symmetric because a site retains the token even if it does not have a request for the CS, which is contrary to the spirit of Ricart and Agrawala's definition of a symmetric algorithm: "No site possesses the right to access its CS when it has not been requested".

Correctness: Mutual exclusion is guaranteed because there is only one token in the system and a site holds the token during the CS execution.

Theorem: A requesting site enters the CS in finite time.

Proof :

- Token request messages of a site S_i reach other sites in finite time.
- Since one of these sites will have token in finite time, site S_i 's request will be placed in the token queue in finite time.
- Since there can be at most $N - 1$ requests in front of this request in the token queue, site S_i will get the token and execute the CS in finite time.

Performance :

- No message is needed and the synchronization delay is zero if a site holds the idle token at the time of its request.
- If a site does not hold the token when it makes a request, the algorithm requires N messages to obtain the token. Synchronization delay in this algorithm is 0 or T .

SINGHAL'S HEURISTIC ALGORITHM

Q.12. Discuss Singhal's heuristic algorithm in detail.

Ans. Singhal's heuristic algorithm :

- In Singhal's heuristic algorithm each site maintains information about the state of other sites in the system and uses it to select a set of sites that are likely to have the token.
- The site requests the token only from these sites, reducing the number of messages required to execute the CS.
- When token request messages are sent only to subset of sites, it is necessary that a requesting site sends a request messages to a site that either holds the token or is going to obtain the token in the near future.
- Otherwise, there is a potential for deadlocks or starvation.
- Thus one design requirement is that a site must select a subset of site such that at least one of those sites is guaranteed to get the token in near future.

Data structure :

- A site S_i maintains two arrays, viz., $SV_i[1...N]$ and $SN_i[1...N]$, to store the information about sites in the system. These arrays store the state and the highest known sequence number for each site, respectively.
- Similarly, the token contains two such arrays as well (denoted by $TSV[1...N]$ and $TSN[1...N]$).

- Sequence numbers are used to detect outdated requests. A site can be in one of the following states :
 R - Requesting the CS
 E - Executing the CS
 H - Holding the idle token
 N - None of the above
- The arrays are initialized as follows :
 For every site $S_i, i = 1...N$ do
 $\{SV_i[j] := N \text{ for } j = N...i; SV_i[j] := R \text{ for } j = i-1...1;$
 $SN_i[j] := 0 \text{ for } j = 1...N\}$
 Initially, site S_1 is in state H (i.e. $S_1[1] := H$)
 For the token $\{TSV[j] := N \text{ and } TSN[j] := 0 \text{ for } j = 1...N\}$
- Note that arrays $SV[1, ..N]$ of sites are initialized such that for any two sites S_i and S_j , either $SV_i[j] = R$ or $SV_j[i] = R$.
- Since the heuristic selects every site that is requesting the CS according to local information (i.e. the SV array), for any two sites that are requesting the CS concurrently, one will always send a token request message to the other.
- This ensures that sites are not isolated from each other and a site's request message reaches a site that either holds the token or is going to get the token in near future.

The algorithm :**(I) Requesting the critical section :**

- (1) If the requesting site S_i does not have the token, then it takes the following actions :
 - It sets $SV_i[i] = R$.
 - It increments $SN_i[i] := SN_i[i] + 1$
 - It sends REQUEST (i, s_n) message to all site S_j for which $SV_i[j] = R$, (s_n is the updated value of $SN_i[i]$)
- (2) When a site S_j receives the REQUEST (i, s_n) message, it discards the message if $SN_j[i] \geq s_n$ because the message is out dated. Otherwise, it sets $SN_j[i]$ to ' s_n ' and takes the following actions based on its own state :

- $SV_i[j] = N$: Set $SV_i[i] := R$
 - $SV_i[j] = R$: If $SV_i[i] \neq R$, then set $SV_i[i] := R$ and send a REQUEST ($j, SN_i[j]$) message to S_i (else do nothing).
 - $SV_i[j] = E$: Set $SV_i[i] := R$
 - $SV_i[j] = H$: Set $SV_i[i] := R$, $TSV[i] := R$, $TSN[i] := s_n$, $SV_i[j] := N$ and send the token to site S_i .
- (II) Executing the critical section :
- S_i executes the CS after it has received the token. Before entering the CS, S_i sets $SV_i[i]$ to E.
- (III) Releasing the critical section :
- (I) Having finished the execution of the CS, site S_i sets $SV_i[i] = N$ and $TSV[i] := N$ and updates its local and token vectors in the following way :
- For all $S_j, j = 1$ to N do
- if $SN_i[j] > TSN[j]$
- then
- (* update token information from local information*)
- { $TSV[j] := SV_i[j]$; $TSN[j] := SN_i[j]$ }
- else
- (* update local information from token information*)
- { $SV_i[j] := TSV[j]$; $SN_i[j] := TSN[j]$ }
- (2) If $(\forall j :: SV_i[j] = N)$, then set $SV_i[i] := H$, else send the token to a site S_j such that $SV_i[j] = R$.
- The fairness of the algorithm depends upon the degree of fairness with which a site is selected for receiving the token, after a site is finished with the execution of its CS. Ideally, the token should not be granted to a site twice or more, while other sites are waiting for the token.
 - When a site requests access to the critical section, it sends request messages to all the sites which, according to its local state information are also currently requesting the CS.
 - The central ideal behind the algorithm is that a site's request for the token reaches a site that has the token even though the site does not send request messages to all sites.

- This is a consequence of the following two factors : (1) How state vectors are initialized and updated and (2) How the sites are selected to send token request message.
- A site updates its state information (arrays SN and SV) from the request messages it receives from other sites and from the information in the token that it receives for critical section access.
- A site S_i sets $SV_i[j]$ to R when it receives a current request message from site S_j or when it receives the token, which has more up-to-date information about site S_j and $TSV[j] = R$.
- Site S_i sets $SV_i[j]$ to N when it receives the token, which has more up-to-date information about site S_j and $TSV[j] = N$. Note that if at site S_i , $SN_i[j] > TSN[j]$, then S_i has more up-to-date information about site S_j , otherwise the token has more up-to-date information about site S_j .
- Since a site does not send request messages to all sites in the system and a site does not send messages to cancel its request messages, the token plays an important role in the dissemination of system state information.

Correctness :

Theorem : A requesting site enters the CS in finite time.

Proof :

- Even though a requesting site does not send token request messages to all other sites, its token request message reaches a site that has the token in finite time.
- When this site updates the token vector, entry for the requesting site is set to R and consequently, the requesting site gets the token in finite time.

Performance :

- A salient feature of this algorithm is that a site can access the critical section without communicating with every site in the system.
- In low to moderate loads, the average message traffic is $N/2$ because each site sends REQUEST messages to half the sites on average.
- It increases to N at high loads as most sites will be requesting the CS (which is reflected at site S_i by $SV_i[j] = R$ for most j 's).

- The synchronization delay in this algorithm is T . An interesting feature of this algorithm is that it adapts itself to the environment of non uniform traffic of CS requests and to statistical fluctuations in the traffic of CS requests to further reduce the number of messages exchanged.
- The algorithm does not have any additional message overhead for the dissemination of state information, except for a slightly larger token message (which is passed comparatively infrequently). Since entries in the token state array (TSV) are either R or N, this array can be a binary array.

RAYMOND'S TREE-BASED ALGORITHM

Q.13. Explain Raymond's tree-based algorithm.

Ans. Raymond's tree-based algorithm :

- In Raymond's tree based algorithm, sites are logically arranged as a directed tree such that the edges of the tree are assigned direction toward the site (root of the tree) that has the token.
- Every site has a local variable holder that points to an immediate neighbour node on a directed path to the root node.
- Thus, holder variables at the sites define logical tree structure among the sites.
- If we follow holder variables at sites, every site has a directed path leading to the site holding the token.
- At root site, holder points to itself.
- Every site keeps a FIFO queue, called request_q which stores the request of those neighbouring sites that have sent a request to this site, but have not yet been sent the token.

Algorithm :

- (I) **Requesting the critical section :**
 - (1) When a site wants to enter the CS, it sends a REQUEST message to the node along the directed path to the root, provided it does not hold the token and its request_q is empty. It then adds its request to its request_q. (Note that a nonempty request_q at a site indicates that the site has sent a REQUEST message to the root node for the top entry in its request_q).
 - (2) When a site receives a REQUEST message, it places the REQUEST in its request_q and sends a REQUEST message along the directed

path to the root provided it has not sent out a REQUEST message on its outgoing edge (for a previously received REQUEST on its request_q).

- (3) When the root site receives a REQUEST message, it sends the token to the site from which it received the REQUEST message and sets its holder variable to point at that site.
- (4) When a site receives the token, it deletes the top entry from its request_q, sends the token to the site indicated in this entry and sets its holder variable to point at that site. If the request_q is nonempty at this point, then the site sends a REQUEST message to the site which is pointed at by holder variable.

(II) **Executing the critical section :**

A site enters the CS when it receives the token and its own entry is at the top of its request_q. In this case, the site deletes the top entry from its request_q and enters the CS.

(III) **Releasing the critical section :**

After a site has finished execution of the CS, it takes the following actions:

- (1) If its request_q is nonempty, then it deletes the top entry from its request_q, sends the token to that site and sets its holder variable to point at that site.
- (2) If the request_q is nonempty at this point, then the site sends a REQUEST message to the site which is pointed at by the holder variable.

Correctness :

The algorithm is free from deadlocks because the acyclic nature of tree configuration elements the possibility of circular wait among requesting sites.

Theorem :

A requesting site enters the CS in finite time.

Proof :

- A formal correctness proof is long and complex. Thus, an informal correctness proof is provided.
- The essence of proof is based on the following two facts : (1) A site serves requests in its request_q in the FCFS order and (2) Every site has a path leading to the site that has the token.

- Due to the latter fact and step 2 of the algorithm, when a site S_i is making a request, there exists a chain of requests from site S_i to site S_h , which holds the token.
- Let the chain be denoted by $S_i, S_{i1}, S_{i2}, \dots, S_{ik-1}, S_{ik}, S_h$. When S_h receives a REQUEST message from S_{ik} , it sends the token to S_{ik} .
- There are two possibilities: S_{ik-1} 's request is at top of S_{ik} 's request_q or it is not at the top.
- In the first case, S_{ik} sends the token to site S_{ik-1} . In the second case, S_{ik} sends the token to the site, say S_j , at the top of its request_q and also sends it REQUEST message (see Step 4 of the algorithm). This extends the chain of requests to $S_i, S_{i1}, S_{i2}, \dots, S_{ik-1}, S_{ik}, S_j, \dots, S_l$, where site S_l executes the CS next.
- Note that due to fact (1) above, all the sites in the chain $S_j, \dots, S_l$ will execute the CS at most once before the token is returned to site S_{ik} . Thus, site S_{ik} sends the token to S_{ik-1} in infinite time.
- Likewise, S_{ik-1} sends the token to S_{ik-2} in finite time and so on. Eventually, S_{i1} sends the token to S_i . Consequently, a requesting site eventually receives the token.

Example :

- In Fig.(1) site S_5 sends a REQUEST message to S_2 , which propagates it to the root S_1 .

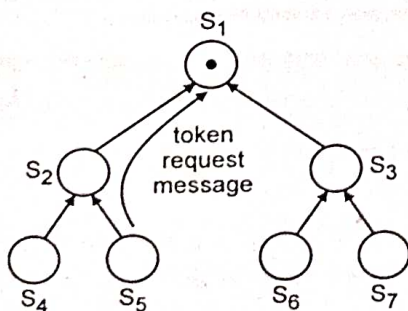


Fig. (1) Site S_5 is requesting CS

- In Fig.(2), Root S_1 sends the token to S_2 which in turn send the token to S_5 and it also reverse the direction of the edges on path.

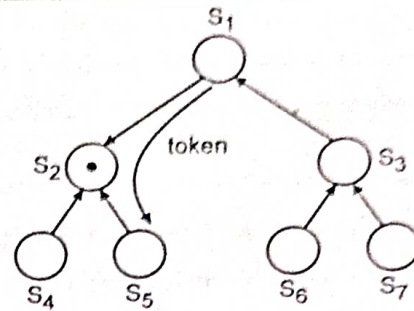


Fig. (2) The token is in transit to S_5

- In Fig.(3), the site that executes the CS last becomes the new root.

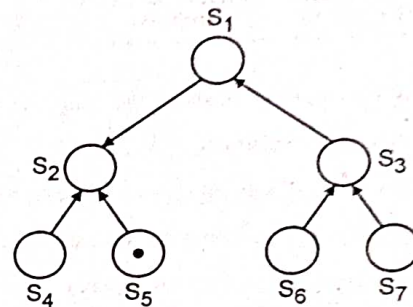


Fig. (3) State after S_5 has received to token

Performance :

- The average message complexity of Raymond's algorithm is $O(\log N)$ because the average distance between any two nodes in a tree with N nodes is $O(\log N)$.
- Synchronous delay in this algorithm is $(T \log N)/2$ because the average distance between two site to successively execute the CS is $(\log N)/2$.

COMPARATIVE PERFORMANCE ANALYSIS

Q.14. Write a short note on response time.

Ans. Response time :

- At low loads, there is hardly any contention among requests for the CS. Therefore, the response time under low load conditions for many algorithms is simply a round trip message delay ($= 2T$) to acquire permission or token plus the time to execute the CS ($= E$).
- In Raymond's algorithm, the average distance between a requesting site and the site holding the token is $(\log N)/2$. Thus, the average round trip delay to acquire the token is $T (\log N)$.

- As the load is increased, response time increases in all mutual exclusion algorithms because contention for access to the CS increases.
- Different algorithms see different increases in response time with respect to load. A closed form expression of response time as a function of load is not known for these algorithms.

A comparison of performance (l = light load, h = heavy load) :

(I)	NON-TOKEN	Resp. time (l)	Sync Delay	Message (l)	Messages (h)
(i)	Lamport	$2T + E$	T	$3(N - 1)$	$3(N - 1)$
(ii)	Ricart-Agrawala	$2T + E$	T	$2(N - 1)$	$2(N - 1)$
(iii)	Mackawa	$2T + E$	$2T$	$3\sqrt{N}$	$5\sqrt{N}$
(II)	TOKEN	Resp. time (l)	Sync Delay	Message (l)	Messages (h)
(i)	Suzuki and Kasami	$2T + E$	T	N	N
(ii)	Singhal's Heuristic	$2T + E$	T	$N/2$	N
(iii)	Raymond	$T(\log N) + E$	$T(\log N) / 2$	$\log(N)$	4

Q.15. Write a short note on synchronization delay.

Ans. Synchronization delay :

- The synchronization delay is the delay due to the sequential message exchanges required after a site leaves the CS and before the next site enters the CS.
- In many of algorithms, a site exiting the CS directly sends a REPLY message or the token to the next site to enter the CS, resulting in a synchronization delay of T .
- Mackawa's algorithm has a synchronization delay of $2T$ because a site exiting the CS unlocks an arbiter site (by sending it a RELEASE message) and then the arbiter site sends a GRANT message to the next site to enter the CS (thus, subject to two serial message delays).
- In Raymond's algorithm, two sites that consecutively execute the CS can be located at any position in the tree. The token propagates serially along the edges in the tree to the site to enter the CS next.

- Since the average distance between two nodes in a typical tree with N nodes is $(\log N)/2$, the synchronization delay in this algorithm is $T(\log N)/2$.
- However, when structure-based token algorithms (such as Raymond) use a greedy strategy, synchronization delay in heavy loads becomes T .

Q.16. Write a short note on message traffic.

Ans. Message traffic :

- The Lamport's Ricart-Agrawala and Suzuki-Kasami algorithms respectively require $3(N - 1)$, $2(N - 1)$ and N messages per CS execution, irrespective of the load.
 - In other algorithms, the number of messages needed depends upon the load and is discussed below :
- (1) Light load :
 - Raymond's algorithm requires $\log N$ messages per CS execution. This is because a request for a token must travel along the edges of the tree from the requester node to the root node and the token must travel back from the root node to the requester node.
 - Note that the average distance between the root and a node in a typical tree with N nodes is $(\log N)/2$.
 - Singhal's heuristic algorithm requires $N/2$ message per CS execution under light load conditions.
 - (2) Heavy load :
 - In Raymond's algorithm, message traffic decreases as load increases due to the sharing of request messages at high load, when a site receive a request message from another site, it does not forward it if it has already sent a request message.
 - Under heavy load conditions, message traffic in Singhal's heuristic algorithm degenerates to that in the Suzuki-Kasami algorithm and requires N messages per CS execution.
 - (3) Load nonuniformly distributed over sites :
 - When the rate of CS requests is nonuniformly distributed over sites, Singhal's heuristic algorithm has the potential to substantially reduce the message traffic by adapting to the prevailing system state.
 - Likewise, piggybacking and greedy strategies can substantially reduce message traffic in Raymond's algorithm provided requests for the CS are localized to a few neighbouring sites.

- For example, with these strategies, the message traffic in Raymond's algorithm under heavy load conditions is four message per CS execution.

Q.17. Write a short note on universal performance bounds.

Ans. Universal performance bounds :

- Due to their idiosyncrasies, different mutual exclusion algorithm have widely varying performance characteristics. Nevertheless we can determine universal performance bounds for mutual exclusion algorithms.
- These bounds depend upon system characteristics, not upon any particular mutual exclusion algorithm.

Universal performance bounds :

Sr. No.	Metrics	Bound	Explanation
(1)	Minimum synchro-nization delay	T	At least one message has to be transferred before the next site can receive the notification of permission to enter the CS.

(2)	Maximum throughput	$1/(T + E)$	The minimum time interval between two successive CS execution is $T + E$
(3)	Minimum response time	$2T + E$	One round trip message delay to acquire permission or the token plus time to execute the CS.
(4)	Maximum average response time	$N(T + E)$	This occurs at heavy loads (when all sites always have pending requests). Note that system executes requests at a rate $r = 1/(T + E)$. From Little's Law, response time $= N / r = N(T + E)$

POINTS TO REMEMBER:

- Mutual exclusion is a fundamental problem in distributed computing system ensures that concurrent access of processes to a shared resource or data is serialized i.e. executed in a mutually exclusive manner.
- The token based mutual exclusion algorithms are in general more message-efficient than non-token based mutual exclusion algorithms.
- A single token based message is returned instead of individual reply messages from number of sites.